

Breaking the Speed Limit

Wenxin Jiang Jian Yin

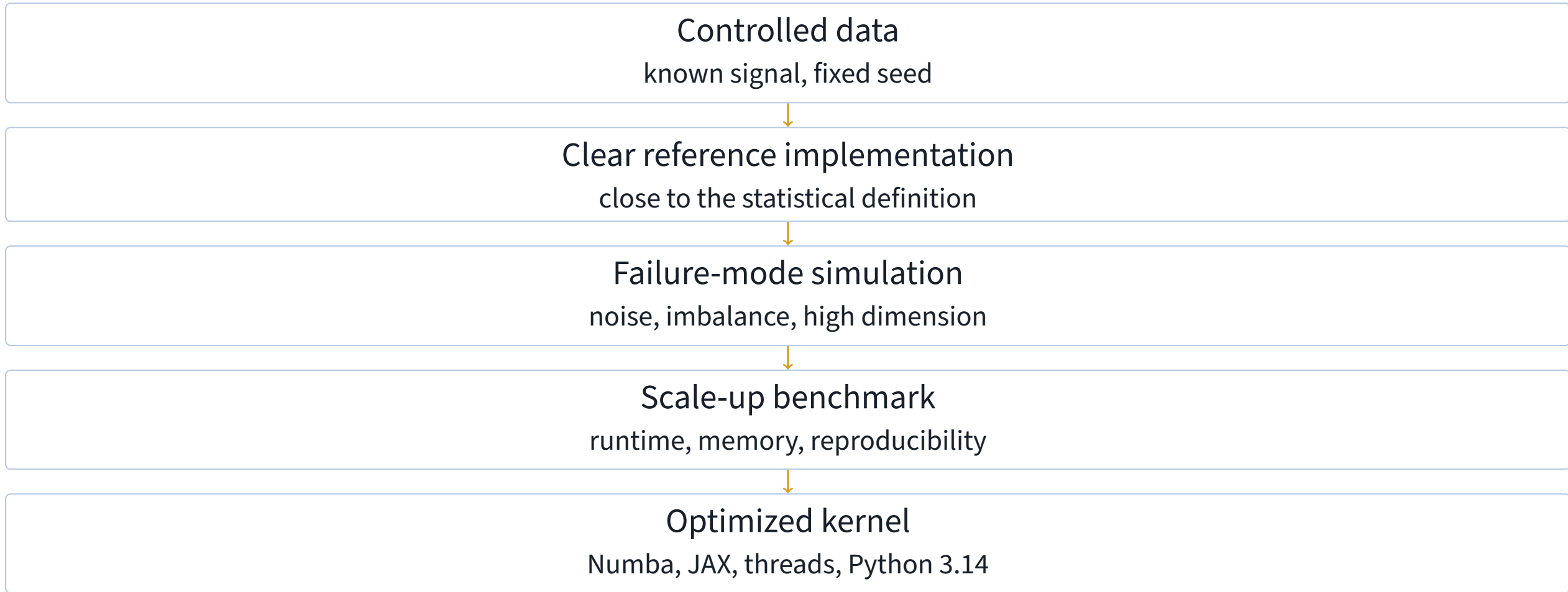
Department of Biostatistics, City University of Hong Kong PyCon US 2026

A workflow talk disguised as a performance talk

Statistical code does not usually start with the fastest kernel. It starts with a question: **can I trust this result?**

We use simulation to build trust, then optimize only the computational pattern that simulation proves is expensive.

The simulation loop



Why this matters

- A faster implementation that changes the statistic is a different method.
- Big data can hide mistakes that small controlled simulations reveal.
- The right tool is the one that preserves statistical meaning while removing the real bottleneck.

Two workloads

k-means: iterative pressure

Lloyd’s algorithm repeats assignment and centroid updates. It exposes Python loop overhead, temporary arrays, cold-start compilation, and high-dimensional scaling.

Permutation test: resampling pressure

The statistic is simple, but R independent resamples expose random generation cost, process copies, shared memory, and thread-level parallelism.

Local evidence

10.8×

Numba over naive NumPy for k-means at $N = 10^6$

0.17 s

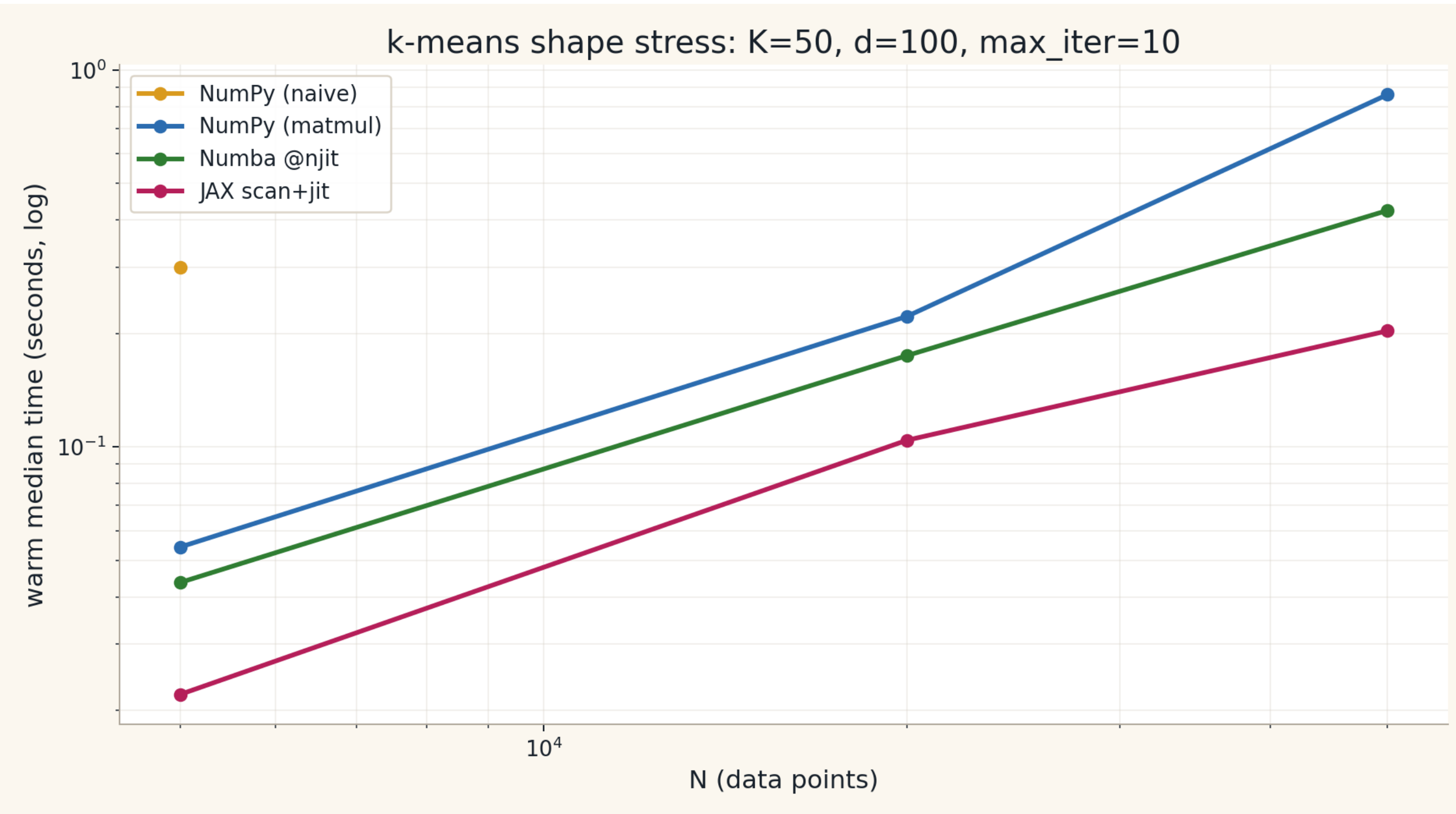
Free-threaded ThreadPool at $R = 10,000$

833 MiB

Child-process RSS in multiprocessing

Current local env: Python 3.12.2, NumPy 1.26.4, Numba 0.59.1, JAX 0.4.25 on CPU. Python 3.14t is confirmed free-threaded.

What changed when K and d grew



At $K = 50$, $d = 100$, textbook broadcasting hits an $O(NKd)$ temporary. The matrix-multiply form becomes useful, and JAX/Numba lead the CPU story.

Three memorable findings

1. Validate before optimizing

The k-means implementations share fixed initial centroids; a sklearn fixed-init check matches inertia to relative error $\sim 1.5 \times 10^{-16}$.

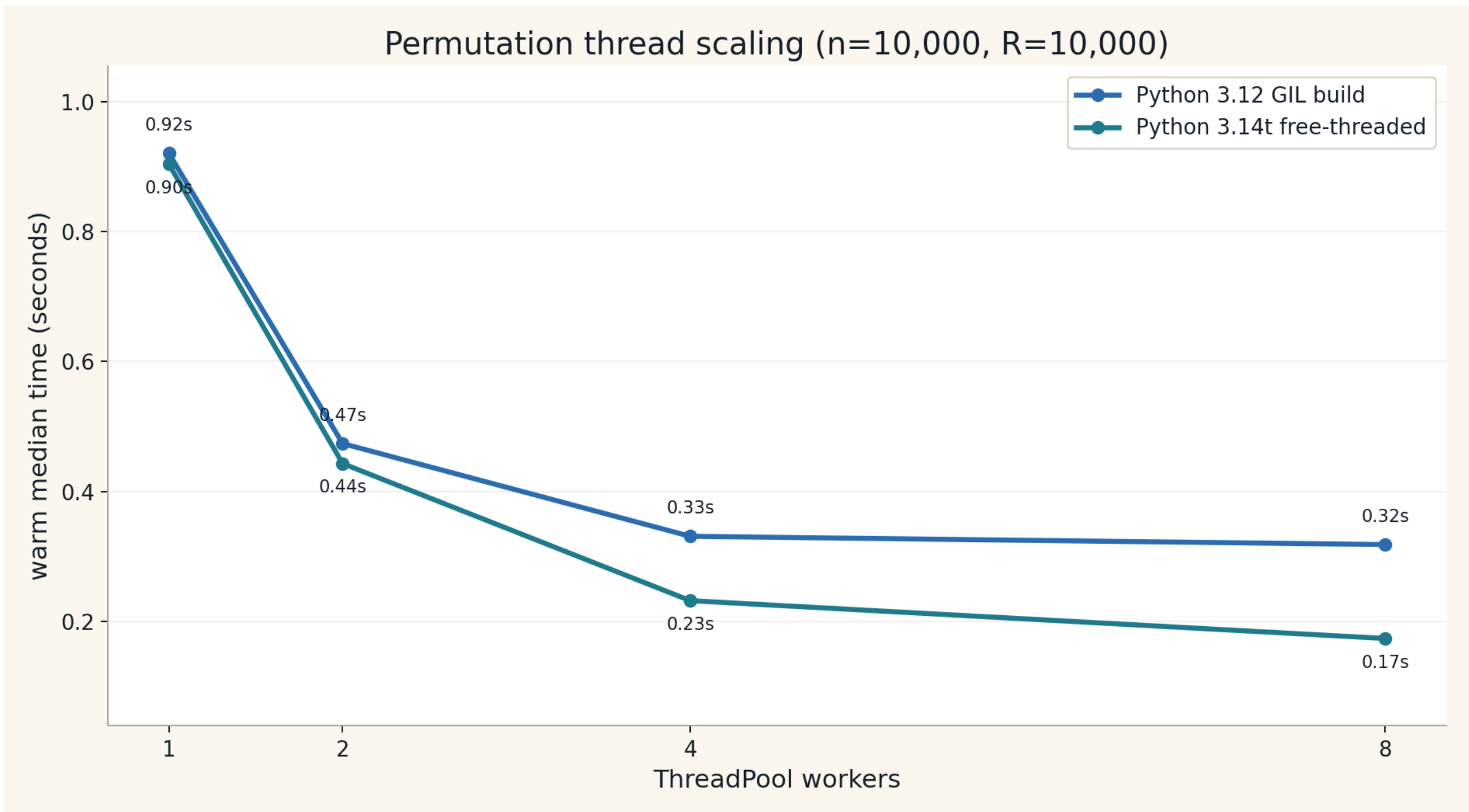
2. Shared memory is a feature

For permutation tests, threads and Numba keep one in-process input array. Multiprocessing pays startup cost and carries worker copies.

3. JAX is not magic CPU parallelism

JAX is excellent when the workload maps to XLA or accelerators. CPU vmap over random permutations is an honest anti-pattern here.

Free-threaded Python: same thread code, different ceiling



The local Python 3.14t no-GIL build improves the same ThreadPool permutation code from 0.32 s to 0.17 s at 8 workers.

Take the workflow with you

Clear enough to trust. Fast enough to scale.

github.com/lucajiang/FastStatisticalModels4Python

Decision guide

Start with trust

Controlled data, fixed seeds, and a reference implementation define the statistic before any optimization changes the code.

Fuse hot loops

When Python owns the inner loop, Numba is the most direct path to compiled scalar logic without changing the method.

Watch memory shape

Broadcasting can create the right answer through the wrong temporary. Matrix identities and fused kernels reduce the real bottleneck.

Choose shared data

For resampling, threads and free-threaded Python keep one input array; processes need an explicit memory budget.